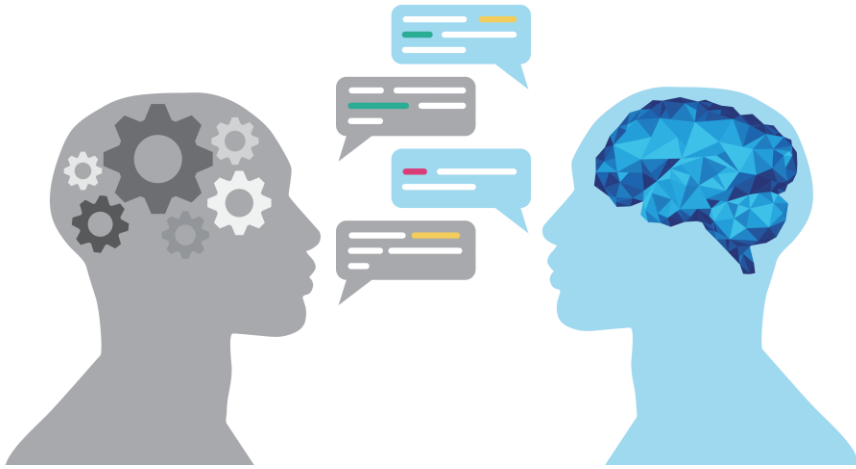




Road to ChatGPT: Basics of AI in Language Processing Day 2

A/Prof Edmund Low
National University of Singapore
edmundlow@nus.edu.sg

Recap – Day 1

- Introduction to AI and Deep Learning
- Building our very first neural network
- How neural networks work

Agenda

- Introduction to Natural Language Processing
- Cleaning our text data
- Tokenization
- Transforming sentences into sequences
- Padding

Let's open up our notebook from Day 1

RTCGPT-Class1

Notebook Input Output Logs Comments (0) Settings

0

Edit

⋮

Add Tags

We will build our very first neural network model here

In [2]:

```
# Importing the necessary libraries
# What's tensorflow, keras and Numpy? Let's go to the slides

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.layers import Dense
import numpy as np

# Say we wish to find out the relationship between our inputs, xs, and outputs, ys
# So that we can predict ys from xs
# This is quintessentially what ML is!
# But of course, this is a very simple example
# Since deep learning is a type of ML, we can use neural networks to help us

# Can we guess what the actual relationship is? Let's go to the slides
```

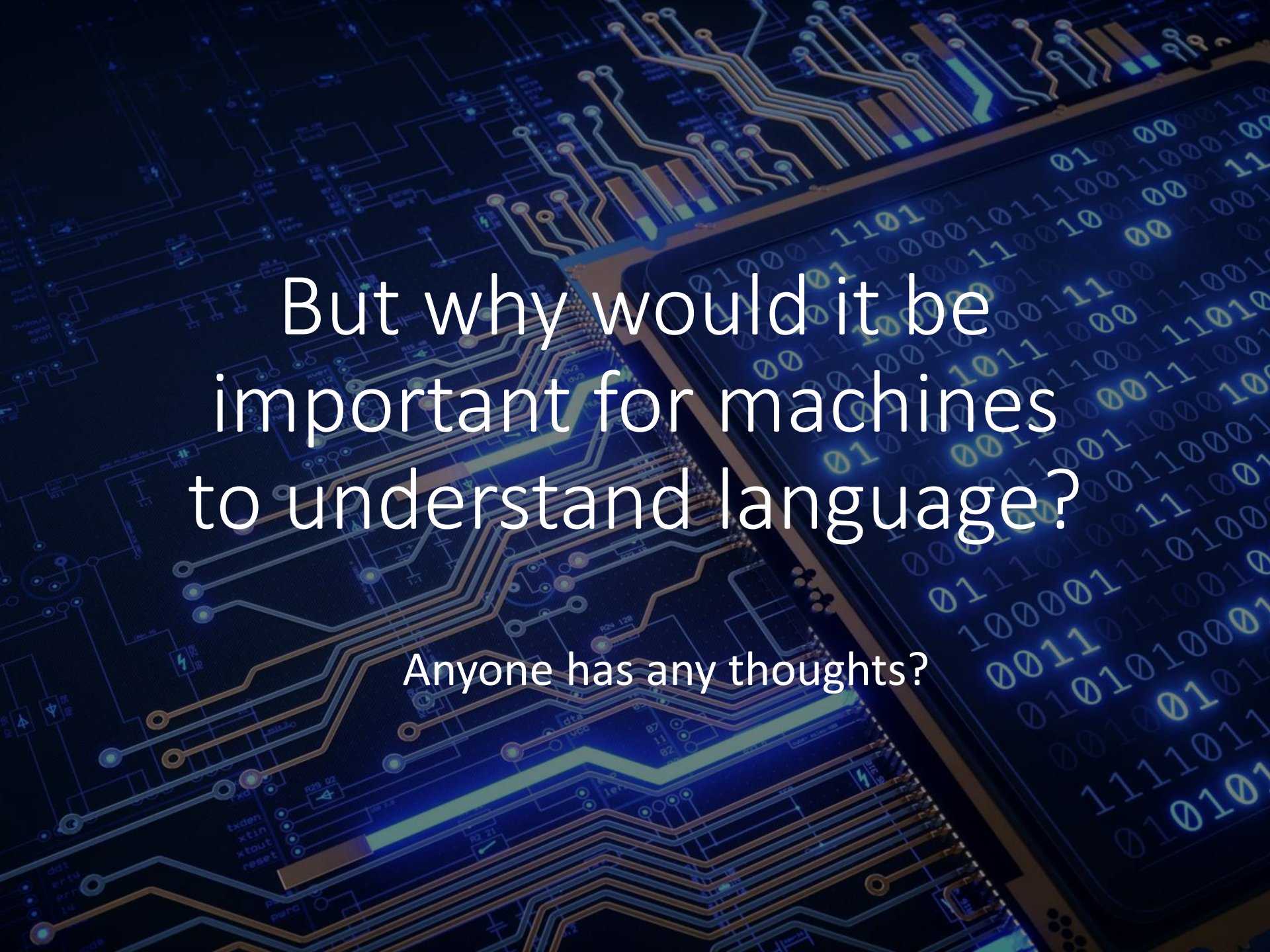
Table of Contents

>

| Visualising our notebook

Making predictions

We will notice that our headings (those markdown cells with a single “#”) are now bookmarked on the RHS panel

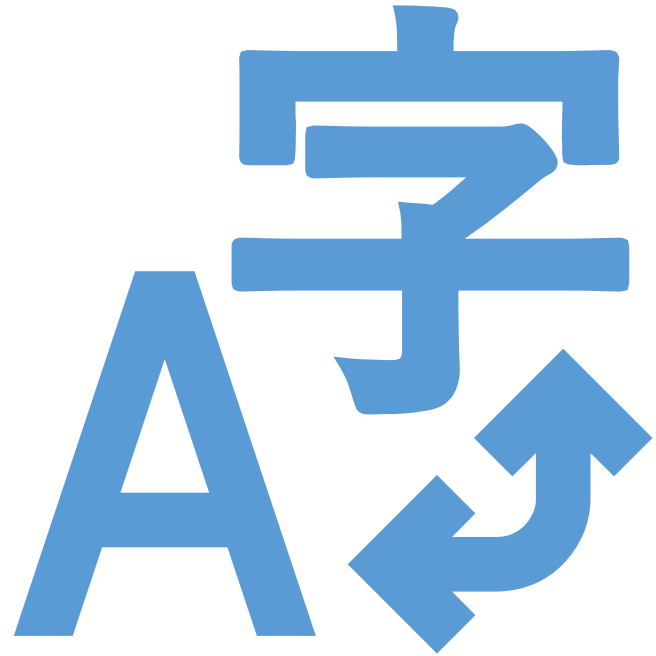


But why would it be
important for machines
to understand language?

Anyone has any thoughts?

Challenges in NLP

- But the ability to “understand” natural language has proven to be quite a challenge to machines
- Early attempts at NLP involves writing / hard-coding complex sets of rules to model language
 - E.g. if(apple) = 苹果
- But what would be the problem here?



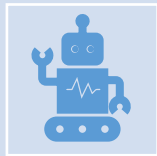
Challenges in NLP

- Open up a notebook on Kaggle and enter the following in a code cell: `print('Hello world')`
- Now change the code to: `printt('Hello world')`
 - What do we notice?
- So what we can draw from this?

Challenges in NLP

- For computer languages : rules came first, then usage
- But for human language, usage came first, then rules of linguistics. Which is why we can tolerate variations from the rules
- At the same time this also means that natural language is not as pliable to formalization

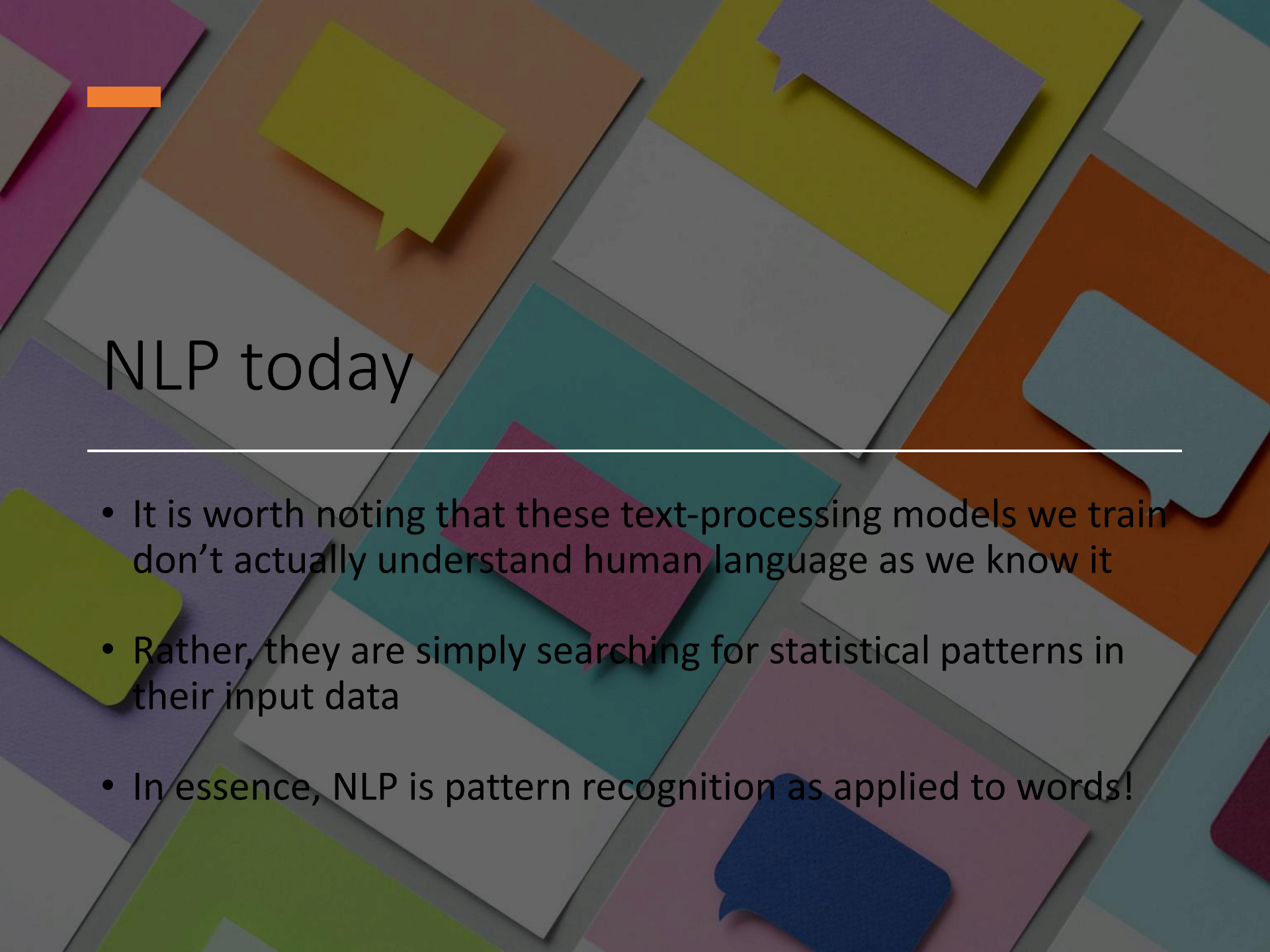
NLP today



Modern NLP, therefore leverages on machine learning
(and most commonly these days, deep learning)



To learn from input language datasets so as to make
predictions, e.g. on sentiment, translations,
generating text

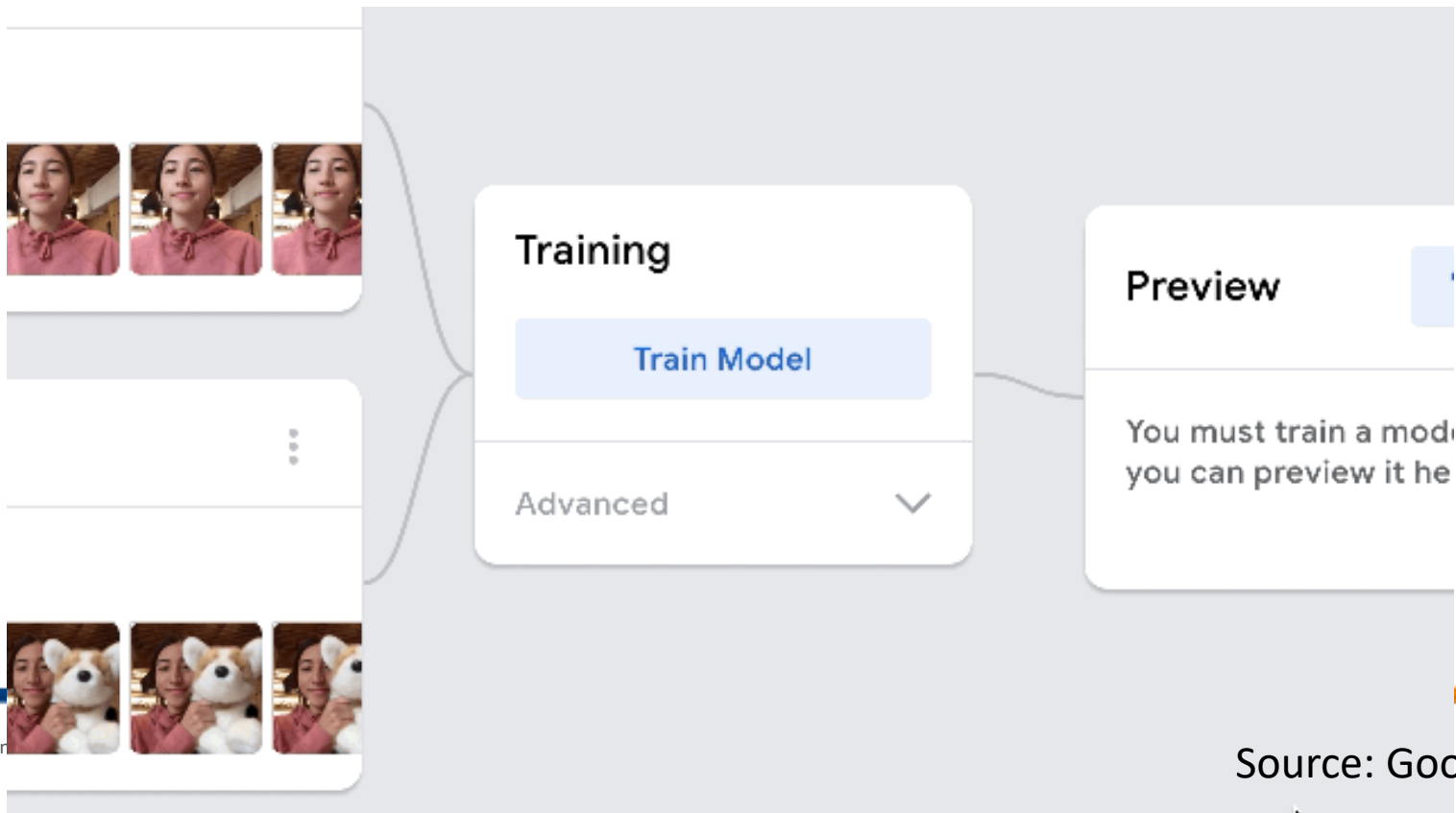


NLP today

- It is worth noting that these text-processing models we train don't actually understand human language as we know it
- Rather, they are simply searching for statistical patterns in their input data
- In essence, NLP is pattern recognition as applied to words!

In the same way that we trained our model previously...

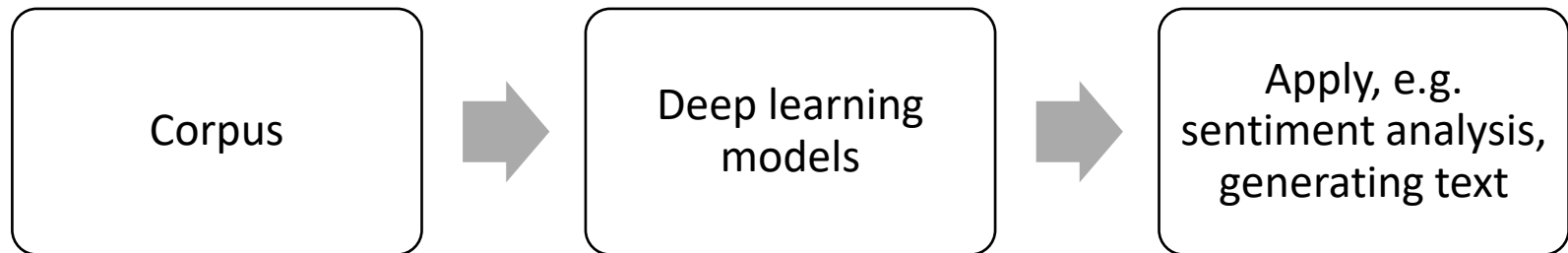
- To recognise patterns in images (person with dog vs person with no dog)



Starting our NLP journey

- Since deep learning models uses differentiable functions, we can't take raw text as inputs
 - Inputs have to be numeric tensors
- Vectorizing text : process of transforming text into numeric tensors
- “I am going to the market today” = [1, 3, 23, 8, 5, 17, 29]

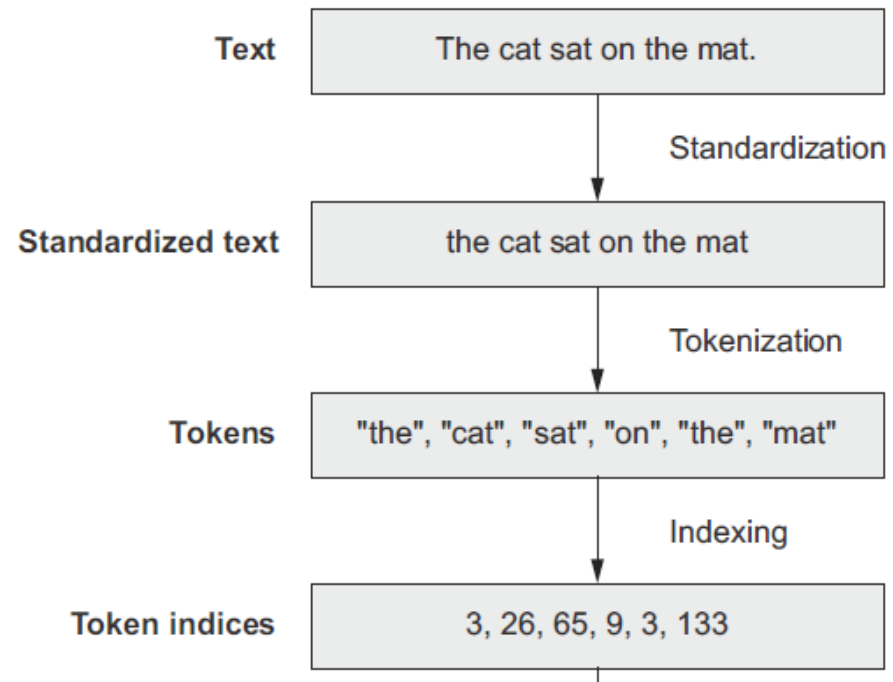
Big picture-wise, this is what we will discuss in our class today



How to prepare / pre-process our text data for learning by the models

The keys steps in text vectorization involve:

Text pre-processing



Let's go through each of these steps

Text standardization

- Let's look at these sentences:

- 1 {
 - “This class is boring. Don’t we wish we are outside playing instead?”
- 2 {
 - “this class is boring; dont we wish we are outside playing instead?”

- How similar are these sentences? What are the differences here? Do they matter in the meaning of the sentence?

Text standardization

- To us, we know that they are (almost) similar, with similar meanings
- But an ML model doesn't know this; it will treat
 - “This” and “this”
 - “boring.” and “boring;”
 - “Don’t” and “dont”

as different words, resulting in different indexings

Text standardization

- So we need some form of standardization, so that similar sentences in our training data will end up having similar numerical representations after vectorization
- Most commonly, we do this by converting to lowercase, and removing punctuation characters

Tokenization

- Once we have standardized our text, we can split them up into units (called tokens) for vectorization
- A common way to tokenize is at the word-level, i.e. each word is a token
- Another approach, called N-gram tokenization, takes groups of N consecutive words, e.g. “is boring” and “outside playing” would be 2-gram tokens

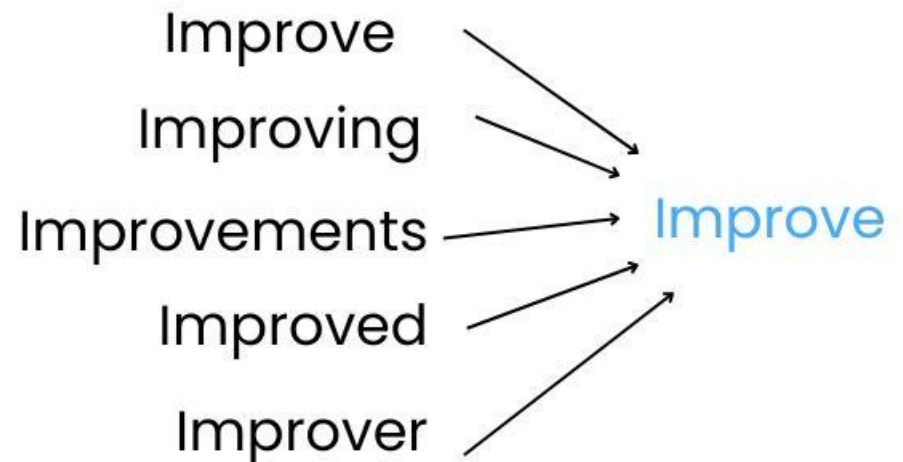
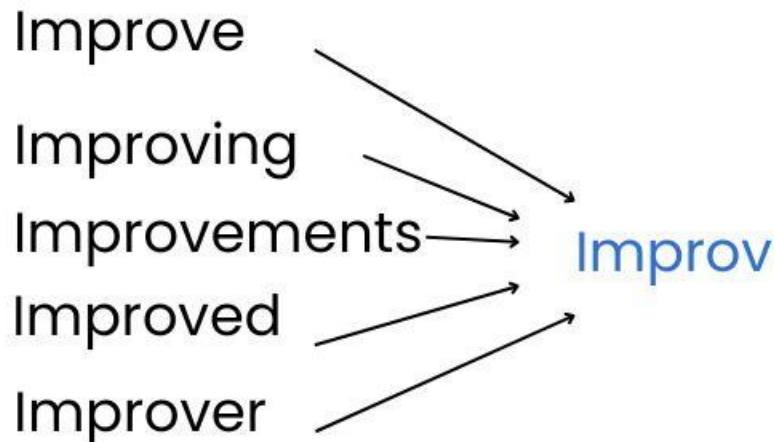
Indexing

- After tokenization, we'll build an index of all the words found in the training data, i.e. our vocabulary
- For each word in our vocabulary, we will assign a unique integer, i.e. indexing

Additional info

- For OpenAI: one token $\sim$ 4 characters in English (including spaces)
- Therefore, the quote “You miss 100% of the shots you don't take” $\sim$ 11 tokens (41 characters)
- Sometimes, in standardization, we might also do stemming or lemmatisation

Stemming vs Lemmatization



Example

- Looking at our previous sentence, after converting to lowercase and removing punctuation,
- We will end up with the following tokens: this, class, is, boring, dont, we, wish, we, are, outside, playing, instead
- Indexing: this = 1, class = 2, is = 3, boring = 4, dont = 5, we = 6, wish = 7, we = 8, are = 9, outside = 10, playing = 11, instead = 12

Indexing

- In our example, our vocabulary is very small
- But practically, a training dataset is going to have a lot more words, and it is common to restrict our vocabulary to only, say, the top 20000 most common ones
- Why do you think this is?

Vectorization

- Finally, we can convert our sentence into a numeric vector (1D tensor)

[this class is boring dont we wish we are outside playing instead]



[1, 2, 3, 4, 5, 6, 7, 6, 8, 9, 10, 11, 12]

Hands-on

- Let's get started on doing some text vectorization
- Go to Kaggle, and import the "Class2-1" notebook
- Give the notebook a name, "Class2-1: Basics of NLP", and let's go through the notebook

Qn 1

- Take a look at our first word index and the second word index
- Did the new sentence change the word index?
- What does we say about the Tokenizer's default behaviour?

Hands-on

- The next step is to vectorize our sentences
- Let's try that by hand. What do you get?
- Go to our notebooks, run the code cell and compare

Qn 2

- What do we notice about the vectorization results? Convert them back into word sentences, what do we have?
- Why do you think this is?
- Do you think this is going to be a common problem when it comes to text vectorization?

OOV

- One way to handle previously unseen words is an out-of-vocabulary (OOV) token, e.g. <OOV> or [UNK]
- We can specify the OOV token in the tokenizer, and it can be any string (just make sure it's not something that will appear anywhere else in your corpus)
- Let's go back to our notebooks to see how this can be done

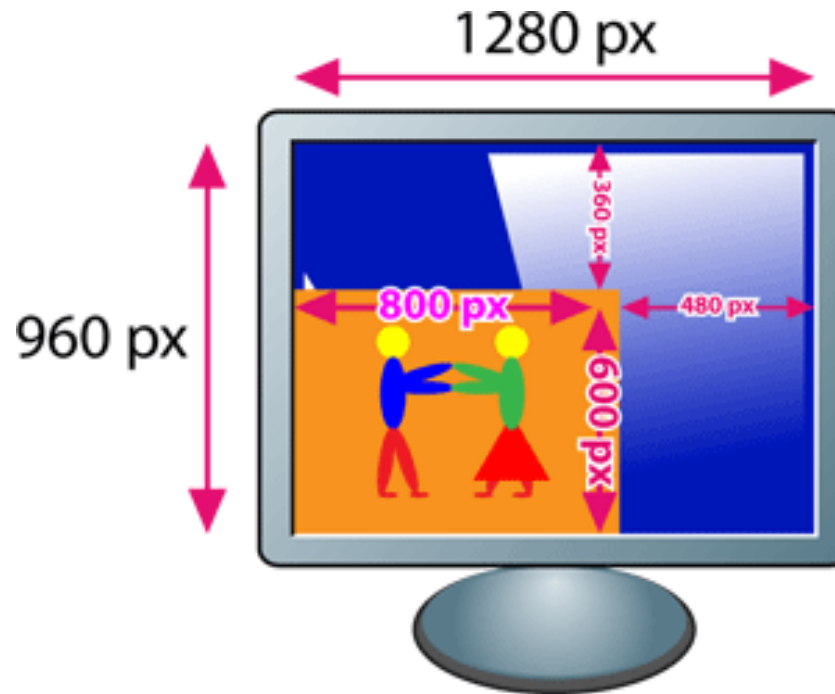
Qn 3

- Convert the vectors back into sentences. What do we have?
- The first sentence is quite close to the original meaning
- The second sentence still lacks quite a bit of context, because most of its words aren't in the corpus, but it's a step in the right direction
 - At least the encoded vectors have the same length as the original test sentences, and we know how many words are OOV

Padding

- When training neural networks, all of our data have to be in the same “shape”
- What does this mean? For instance, if we’re processing image data, some images might be bigger (more pixels) and some smaller (fewer pixels)

Padding



- We need to therefore reformat our images to the same width and height

Padding

- And it's the same with text data as well
- Some sentences have more words; some have fewer words; when we convert to numerical sequences, they are going to have different lengths
- To get them into the same shape, we can use padding – what is padding? Let's go to our notebooks to illustrate

Padding – results discussion

- We notice that the sequences get padded with 0. By default, the index 0 is reserved for the mask token, which means "ignore me, I'm not a word"
- We also notice that the padding, by default, is pre-padding, i.e. the 0's appear at the head of the shorter sentences
- What is the basis of the padding, i.e. how do we know how many "0"s to use for shorter sentences? What's the concern with this basis of padding?
- How to change from pre- to post-padding? Let's return to our notebooks

Qn 4

- What do we notice about the truncating to the max length? Why might this be undesirable?

Removing stopwords

- Sometimes, when we provide text data to our model for learning, there are some words that we **don't want** in the dataset
- These words are known as stopwords, and are those that occur too frequently, and don't add any meaning to the overall text, e.g. **the, and, but**

Removing stopwords

- For instance, when we want to analyse whether a movie review is positive or negative, these words add little to the overall sentiment
- But including them can result in more useful, but less frequently occurring words being omitted when we set a limit to our vocabulary

Removing HTML tags

- Sometimes, text data scraped from the internet includes tags
- For instance, if we go to :
https://www.imdb.com/title/tt0068646/reviews?ref=tt_urv

"It is now past 1 PM and I just finished watching Francis Ford Coppola's "The Godfather". I should probably go to bed. It's late and tomorrow I have to wake up a bit early. But not early enough to postpone writing these lines. Now that I have seen it three times, the opportunity of sharing my thoughts and refreshed insights are too much of a good offer to sit on. So, bear with me." == \$0

"This film works so well because it takes place in an underworld in which we are so embedded that we do not even observe it. Coppola puts us straight in the smack-dab center of what is, admittedly, a society made by criminals for criminals. It is also the reason why it's so welcoming. We are surrounded by its inhabitants--cold-blooded murderers, men who see crime like a 9 to 5 job masquerading as honorable men. And I do mean men. From the outside, we would only witness the horrifying, disturbing manifestations of their well-thought out actions."

"But it goes even deeper than that. It all revolves around the Corleone family led by Don Vito Corleone (Marlon Brando). He is the most honest of these men, sitting right on the edge. But for people like him, who do not fully embrace this world, it's not easy. He avoids conflict until it is absolutely necessary. He is a man defined by moral principles. There is a scene at the beginning, in which, during his daughter's wedding day, one of his associates, Luca Brasi (Lenny Montana) practices his speech that he is going to give to the Don when he meets him. The scene with these two is funny and almost adorable. I could not help but sympathize both of

- We can see that the text comes with associated tags
- These tags don't have any meaning, and should be removed, else they will count towards our vocabulary

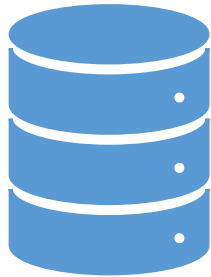
Let's apply what we have seen on an actual dataset

- We will be using the imdb movie reviews dataset from the list of publicly available TensorFlow datasets (TFDS)
- The full catalog is available here:
- <https://www.tensorflow.org/datasets/catalog/overview>

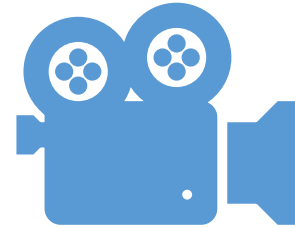
IMDB Reviews Dataset

- https://www.tensorflow.org/datasets/catalog/imdb_reviews
- Dataset comprises 50,000 movie reviews (under the feature “Text”), each of which is labelled either positive or negative in sentiment (under the feature “label”)
- The dataset comprises a train and test split, comprising 25,000 rows each

Predicting sentiment

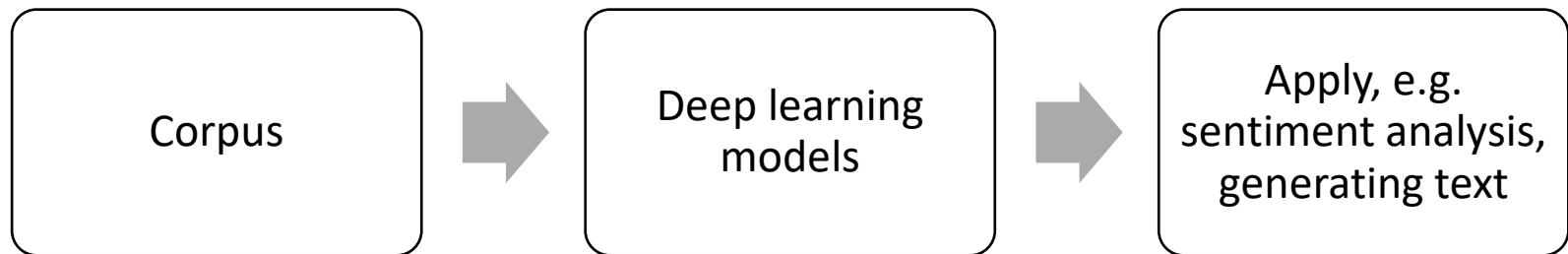


We are going to see if we
can train our neural network,
using this dataset



So that it will be able to
predict the sentiment of new
movie reviews

But first, we have to pre-process our dataset



Preparing our text data
for learning by the
models

Hands-on

- Let's go back to Kaggle, open up a new notebook and import the "Class2-2" notebook
- Rename the notebook, e.g. "RTCGPT Class2-2: Text pre-processing", and let's go through the notebook

Qn 5

- Anything unusual we notice when we check the number of words in our indexed vocabulary?

Qn 6

- Why isn't the limit imposed when we tokenize and index our corpus, but only when we are vectorizing?
- This is to give us the option to change the number of words easily without needing to fit on our corpus again
- Hence, from the results of the vectorization, we can experiment whether the current limit is adequate for our needs, or if we need more words for our task

Qn 6

- Versus a situation where we have to re-generate the word index according to a new limit each time



We vectorize using
20,000 words

20,000 word index



We vectorize using
40,000 words

40,000 word index

Let's go back to our
notebooks to see how
this can be done

TextVectorization layer

- Alternatively, we can work with the Keras TextVectorization layer, which is fast and efficient
- The layer can be dropped directly into a Keras model
- By default, the TextVectorization layer will convert to lowercase, remove punctuation and tokenize by splitting on whitespace

End Day 2 – Part 1